

Reviewer Pack — Minimal Rank of Schur Polynomials in Binary Decision Diagram...

Entry 8c645d7e16b2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 00:01:58 UTC

Minimal Rank of Schur Polynomials in Binary Decision Diagrams vs. Monotone Circuit Satisfiability

Entry ID: 8c645d7e16b2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 00:01:58 UTC

1. Conjecture

Field A (mathematical branch): Schur Polynomials Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Satisfiability

Statement:

[‘For a given n -variables monotone Boolean function f , the minimal rank of its associated Schur polynomial is $\Omega(n \log n)$.’, ‘This minimal rank is preserved under linear substitutions and is independent of the specific variable ordering.’, ‘The minimal rank of the Schur polynomial provides a lower bound on the size of any monotone circuit computing f .’]

Rationale (proposer’s reasoning):

[‘Schur polynomials provide a rich algebraic structure that captures the essence of Boolean functions. Their minimal rank has been studied in representation theory but not extensively in complexity theory.’, ‘The conjecture suggests that the properties of Schur polynomials could shed light on the inherent difficulty of monotone circuit satisfiability, which is a fundamental problem in computational complexity.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9bc60a03ce85a746

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 random seeds, at least 80% of monotone Boolean functions f with $n \leq 40$ have a minimal rank of their associated Schur polynomial in $O(n \log n)$, calculated using standard algorithms like MacWilliams identity or Weyl’s trick.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Schur Polynomials" AND "Minimal Rank" AND "Monotone Circuit Satisfiability" - "Linear Substitutions" IN TITLE OR ABSTRACT AND "Schur Polynomial" AND "Monotone Circuit" - "Lower Bound" ON SIZE OF CIRCUIT AND "Schur Polynomial" AND "Monotone Circuit Satisfiability"

Top relevant hits considered: - [<http://arxiv.org/abs/1810.12657v1>] The effect of a country's name in the title of a publication on its visibility and citability - [<http://arxiv.org/abs/1503.01694v1>] An Algebraic Study of Multivariable Integration and Linear Substitution - [<http://arxiv.org/abs/cs/0207003v1>] Analysis of Titles and Readers For Title Generation Centered on the Readers - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/2105.03706v2>] Upper and lower bounds on the size of $B_k[g]$ sets - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def schur_polynomial(f):
        n = len(f)
        if n == 1:
            return f[0]
        else:
            s = [0] * (n + 1)
            s[0] = 1
            for i in range(n):
                s[i+1] = sum(s[j] * f[i-j] for j in range(i+1))
            return s

    def min_rank(poly):
        n = len(poly) - 1
        rank = 0
        for i in range(1, n + 1):
            if poly[i] != 0:
```

```

        rank += 1
    return rank

n_values = [5, 10, 15, 20, 30, 40]
total_rank = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        f = generate_monotone_boolean_function(n)
        poly = schur_polynomial(f)
        rank = min_rank(poly)
        total_rank += rank
        instances_tested += 1

mean_rank = total_rank / instances_tested
conjecture_holds = mean_rank >= n * math.log2(n) * 0.95
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "min_rank",
    "metric_value": mean_rank,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={seeds[first_failing_seed]}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10130
2	preregistration	ollama_remote	glm4:latest	0	0	5522
3	novelty	ollama_remote	glm4:latest	0	0	4977
4	novelty	ollama_remote	glm4:latest	0	0	5591
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15719
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7307
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8245
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7910
9	judge	ollama_remote	glm4:latest	0	0	12908

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 78309 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/8c645d7e16b2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8c645d7e16b2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/8c645d7e16b2.tar.gz (if generated)